```
> # ================================================================================
  ===
  # TauTimp — Redshift from Spatial Flux Growth Rate
  # Onduscular Theory (Teoria Onduscularа) — Author: Viorel-Mihai Popescu
  # Maple Transcript — Version: May 2026
  # ================================================================================
  ===

  restart;
  with(plots):
  with(PDEtools):
  with(DEtools):

  # --------------------------------------------------------------------------------
  ---
  # SECTION 0: Physical Parameters
  # --------------------------------------------------------------------------------
  ---

  # Fundamental constants
  c_SI := 2.99792458e8:          # m/s, speed of light
  H0_SI := 67.41e3 / 3.085678e22: # s^-1, Hubble constant (Planck 2018)
  H0_kms := 67.41:               # km/s/Mpc

  # Onduscular cosmological radii (SI, metres)
  Rs_SI := 7.7664618e25:         # m  (= 8.20933 Gly)
  R_SI  := 3.9228e22:            # m  (= 4.147 million LY)

  # Conversion factor
  Gly_to_m := 9.4607304725808e24: # 1 Gly = 9.461e24 m

  # Onduscular radii in Gly
  Rs_Gly := Rs_SI / Gly_to_m:
  print("Rs (Gly) =", evalf(Rs_Gly, 6));
  # => 8.20933 Gly

  TwoRs_Gly := 2 * Rs_Gly:
  print("2Rs (Gly) =", evalf(TwoRs_Gly, 6));
  # => 16.4187 Gly

  # Boundary parameters
  r_min_flux := 0.290:   # Gly — inner boundary for muon flux normalization
  r_min_cosm := 0.431:   # Gly — epoch of first star formation

  # Hubble horizon
  RH_Gly := c_SI / (H0_SI * Gly_to_m):
  print("Hubble horizon R_H (Gly) =", evalf(RH_Gly, 6));
  # => 14.505 Gly
```

```
# ------------------------------------------------------------------------
---
# SECTION 1: Onduscular Wave Equation and Separation of Variables
# ------------------------------------------------------------------------
---
# Governing equation (spherical symmetry):
#   (Rs/r) * (f_rr + 2/r * f_r) = (1/c^2) * f_tt
#
# Separating f(r,t) = g(r)*u(t):
#   Time ODE:  u'' + lambda^2 * c^2 * u = 0
#   Radial ODE: g'' + (2/r)*g' + (lambda^2 * r / Rs) * g = 0

# Display the radial ODE symbolically
r_var := 'r': Rs_sym := 'Rs': lam := 'lambda_n':
radialODE := diff(g(r_var), r_var, r_var)
            + (2/r_var) * diff(g(r_var), r_var)
            + (lam^2 * r_var / Rs_sym) * g(r_var) = 0:
print("Radial ODE:", radialODE);

# Substitution g(r) = r^(-1/2) * phi(r) reduces to the Airy equation:
#   d^2 phi / d xi^2 = xi * phi,   xi = (lambda^2 / Rs)^(1/3) * r
print("Airy equation: d^2 phi/d xi^2 = xi * phi");
print("Airy zeros a_1 = -2.3381, a_2 = -4.0879, a_3 = -5.5206");

# Eigenvalue quantization from boundary condition Ai(alpha_n * 2*Rs) = 0:
airy_zeros := [-2.3381, -4.0879, -5.5206]:
lambda_n := proc(n)
  local an;
  an := airy_zeros[n];
  return (-an)^(3/2) / (2 * sqrt(Rs_SI));
end proc:

for n from 1 to 3 do
  print(sprintf("lambda_%d = %.6e s^-1 m^-1", n, lambda_n(n)));
  print(sprintf("omega_%d  = %.6e rad/s", n, lambda_n(n) * c_SI));
end do:

# ------------------------------------------------------------------------
---
# SECTION 2: Muon Neutrino Flux P(r,0) — Polynomial Solution
# ------------------------------------------------------------------------
---

# Define symbolic variables (Gly units)
r := 'r':
Rs := Rs_Gly:
r0 := r_min_flux:   # 0.290 Gly (inner boundary, flux normalization)

# Factored form (satisfies boundary conditions)
```

```
P0 := C_norm * (r - r0) * (2*Rs - r):
print("P(r,0) =", P0);

# Compute analytic integral G
integrand_G := (r - r0) * (2*Rs - r) / r^2:
G_analytic := (r0 + 2*Rs) * ln(2*Rs / r0) - 2*(2*Rs - r0):
G_val := evalf(G_analytic * Gly_to_m, 7):
print("G (SI, m) =", G_val);
# Expected: 3.3285e26 m

# Determine normalization constant C from integral F(r) dr = c/H0
RH_SI := c_SI / H0_SI:
print("Hubble horizon R_H (SI, m) =", evalf(RH_SI, 6));

C_val := 4 * Pi * c_SI / (H0_SI * G_val):
print("C =", evalf(C_val, 6), "m^-2");
# Expected: 5.1809 m^-2

# Polynomial coefficients of P(r,0) = C*(r-r0)*(2Rs-r) = -C*r^2 + C*
(r0+2Rs)*r - C*r0*2Rs
# (in Gly units for r, C in m^-2, values in Gly^2 * m^-2 = dimensionless
ratio)
p2 := -C_val:
p1 := C_val * (r_min_flux + 2*Rs_Gly):
p0 := -C_val * r_min_flux * 2*Rs_Gly:
print("p2 =", evalf(p2, 5));
print("p1 =", evalf(p1, 5), "Gly^-1");
print("p0 =", evalf(p0, 5), "Gly^-2");

# ------------------------------------------------------------------------
---
# SECTION 3: Spatial Flux Growth Rate Lambda(r)
# ------------------------------------------------------------------------
---

# Lambda(r) is the spatial flux per unit surface, integrated to give
comoving distance.
# From the PDE solution at t=0, expanded as Laurent series in r (Gly
units):
#
#   Lambda(r) = A2/r^2 + A1/r + A0
#
# Coefficients (Rs = 8.2, c = 3, normalized units):
A2 := 43.1072:
A1 := -0.6533:
A0 := -0.12012:

Lambda := r -> A2/r^2 + A1/r + A0:
print("Lambda(r) =", A2, "/r^2 +", A1, "/r +", A0);
```

```
# Indefinite integral (antiderivative)
Lambda_int := r -> -A2/r + A1*ln(r) + A0*r:
print("Integral of Lambda(r) dr = -43.10796/r +", A1, "*ln(r) +", A0, "*r
+ const");

# Verify: integral from r_min_cosm to 2*Rs gives 93 Gly
cosm_integral := Lambda_int(2*Rs_Gly) - Lambda_int(r_min_cosm):
print("Integral from 0.431 to 16.419 Gly =", evalf(cosm_integral, 5),
"Gly");
# Expected: ~93 Gly

# ------------------------------------------------------------------------
---
# SECTION 4: Hubble Constant Derivation from Onduscular Parameters
# ------------------------------------------------------------------------
---

# H0 derived analytically:
#   H0 = 4*pi*c / (C * G)
H0_derived := 4 * Pi * c_SI / (C_val * G_val):
H0_derived_kms := H0_derived * 3.085678e22 / 1e3:
print("H0 (derived) =", evalf(H0_derived_kms, 5), "km/s/Mpc");
# Expected: 67.41 km/s/Mpc

# ------------------------------------------------------------------------
---
# SECTION 5: PDE Solution — Full Time Dependence
# ------------------------------------------------------------------------
---

# P(r,t) via d'Alembert:
#   P(r,t) = C * [(r - r_min)(2Rs - r) - c^2 * t^2]
# In Gly/Gyr units: c = 1 Gly/Gyr

c_Gly := 1.0:   # Gly/Gyr (speed of light in natural cosmological units)

P_rt := (r_val, t_val) -> C_val * ((r_val - r_min_flux) * (2*Rs_Gly -
r_val) - c_Gly^2 * t_val^2):
print("P(r,t) = C * [(r - r_min)(2Rs - r) - c^2 * t^2]");
print("P(r,0) check at r=8 Gly:", evalf(P_rt(8.0, 0), 5));
print("P(2Rs,0) check (BC):",     evalf(P_rt(2*Rs_Gly, 0), 5));

# ------------------------------------------------------------------------
---
# SECTION 6: PLOT 1 — P(r,0) Muon Neutrino Flux Profile
# ------------------------------------------------------------------------
---

P0_plot := r -> (r - r_min_flux) * (2*Rs_Gly - r):  # unnormalized shape
```

```
plot1 := plot(P0_plot(r), r = r_min_flux .. 2*Rs_Gly,
  color = "Blue",
  thickness = 2,
  title = "Figure 1: Muon Neutrino Flux P(r,0) — Bell Profile",
  labels = ["r  [Gly]", "P(r,0)  [normalized, Gly^2]"],
  labelfont = [TIMES, BOLD, 11],
  titlefont = [TIMES, BOLD, 12],
  gridlines = true,
  axes = BOXED
):
print(plot1);

# ------------------------------------------------------------------------
---
# SECTION 7: PLOT 2 — Spatial Flux Growth Rate Lambda(r)
# ------------------------------------------------------------------------
---

plot2 := plot(Lambda(r), r = r_min_cosm .. 2*Rs_Gly,
  color = "Red",
  thickness = 2,
  title = "Figure 2: Spatial Flux Growth Rate Lambda(r) — TauTimp
Redshift",
  labels = ["r  [Gly]", "Lambda(r)  [Gly^-1]"],
  labelfont = [TIMES, BOLD, 11],
  titlefont = [TIMES, BOLD, 12],
  gridlines = true,
  axes = BOXED,
  view = [r_min_cosm .. 2*Rs_Gly, -1 .. 10]
):
print(plot2);

# ------------------------------------------------------------------------
---
# SECTION 8: PLOT 3 — Cumulative Comoving Distance Integral
# ------------------------------------------------------------------------
---

Cumulative := r -> Lambda_int(r) - Lambda_int(r_min_cosm):

plot3 := plot(Cumulative(r), r = r_min_cosm .. 2*Rs_Gly,
  color = "DarkGreen",
  thickness = 2,
  title = "Figure 3: Cumulative Comoving Distance = Integral of Lambda(r)
",
  labels = ["r  [Gly]", "Cumulative integral  [Gly]"],
  labelfont = [TIMES, BOLD, 11],
  titlefont = [TIMES, BOLD, 12],
  gridlines = true,
```

```
  axes = BOXED
):
print(plot3);

# ------------------------------------------------------------------------
---
# SECTION 9: PLOT 4 — Flux Density at Surface F(r) = P(r,0)/(4*pi*r^2)
# ------------------------------------------------------------------------
---

F_r := r -> C_val * (r - r_min_flux) * (2*Rs_Gly - r) / (4 * Pi * r^2):

plot4 := plot(F_r(r), r = r_min_flux .. 2*Rs_Gly,
  color = "Purple",
  thickness = 2,
  title = "Figure 4: Onduscular Surface Flux Density F(r) = P(r,0)/(4pi
r^2)",
  labels = ["r  [Gly]", "F(r)  [Gly^-3]"],
  labelfont = [TIMES, BOLD, 11],
  titlefont = [TIMES, BOLD, 12],
  gridlines = true,
  axes = BOXED
):
print(plot4);

# ------------------------------------------------------------------------
---
# SECTION 10: PLOT 5 — Lambda(r): Comparison with Hubble Law
# ------------------------------------------------------------------------
---

# Hubble law in natural units: v(r) = H0 * r
# In Gly units: H0 ~ 0.0689 Gyr^-1 ~ constant slope
H0_nat := H0_kms / (3.085678e22 / 1e3 / (3.15576e16)):  # Gyr^-1
print("H0 (Gyr^-1) =", evalf(H0_nat, 4));

Hubble_line := r -> H0_nat * r:

plot5 := plot([Lambda(r), Hubble_line(r)], r = 0.5 .. 2*Rs_Gly,
  color = ["Red", "Navy"],
  thickness = [2, 2],
  linestyle = [SOLID, DASH],
  title = "Figure 5: Lambda(r) vs Linear Hubble Law",
  labels = ["r  [Gly]", "Growth rate  [Gyr^-1]"],
  labelfont = [TIMES, BOLD, 11],
  titlefont = [TIMES, BOLD, 12],
  legend = ["TauTimp Lambda(r)", "Hubble H0*r"],
  legendstyle = [location = right],
  gridlines = true,
```

```
  axes = BOXED,
  view = [0.5 .. 2*Rs_Gly, -0.1 .. 1.0]
):
print(plot5);

# ------------------------------------------------------------------------
---
# SECTION 11: PLOT 6 — P(r,t) Time Evolution (3D Surface)
# ------------------------------------------------------------------------
---

P_shape := (r_val, t_val) -> max(0, (r_val - r_min_flux) * (2*Rs_Gly -
r_val) - t_val^2):

plot6 := plot3d(P_shape(r, t), r = r_min_flux .. 2*Rs_Gly, t = 0 .. 10,
  axes = BOXED,
  labels = ["r [Gly]", "t [Gyr]", "P(r,t) [norm.]"],
  title = "Figure 6: Time Evolution of Muon Neutrino Flux P(r,t)",
  titlefont = [TIMES, BOLD, 12],
  style = surface,
  shading = ZHUE,
  orientation = [45, 65],
  grid = [40, 30]
):
print(plot6);

# ------------------------------------------------------------------------
---
# SECTION 12: PLOT 7 — Redshift z(r) from TauTimp vs Hubble
# ------------------------------------------------------------------------
---

# TauTimp redshift proxy: z(r) proportional to cumulative Lambda integral.
# Cumulative(2*Rs_Gly) ~ 93 Gly; we map this to observed z_max ~ 15 (GN-
z11 scale).
# Normalization factor = z_max / integral_total = 15 / 93 ~ 0.1613
[Gly^-1].
# This is dimensionally correct: [z] = dimensionless, [integral] = Gly.
z_max_ref     := 15.0:
integral_tot  := evalf(Cumulative(2*Rs_Gly), 6):
z_norm_factor := z_max_ref / integral_tot:
printf("Integral_tot (Gly)     = %.4f\n", integral_tot);
printf("z_norm_factor (Gly^-1) = %.6f   [= z_max / integral_tot]\n",
z_norm_factor);

z_TauTimp := r -> z_norm_factor * Cumulative(r):

# Standard Hubble redshift linear approximation (z = H0*r/c, c = 1
Gly/Gyr)
```

```
z_Hubble := r -> H0_nat * r:

plot7 := plot([z_TauTimp(r), z_Hubble(r)], r = 0.5 .. 2*Rs_Gly,
  color = ["OrangeRed", "SteelBlue"],
  thickness = [2, 2],
  linestyle = [SOLID, DASH],
  title = "Figure 7: Redshift z(r) — TauTimp vs Linear Hubble",
  labels = ["r  [Gly]", "Redshift  z"],
  labelfont = [TIMES, BOLD, 11],
  titlefont = [TIMES, BOLD, 12],
  legend = ["TauTimp z(r)", "Hubble linear z(r)"],
  legendstyle = [location = top],
  gridlines = true,
  axes = BOXED
):
print(plot7);

# ------------------------------------------------------------------------
---
# SECTION 13: Numerical Summary
# ------------------------------------------------------------------------
---

print("============================================================");
print("NUMERICAL SUMMARY — TauTimp Redshift (May 2026)");
print("============================================================");
print(sprintf("Rs              = %.6f  Gly", Rs_Gly));
print(sprintf("2Rs             = %.6f  Gly  (onduscular boundary)", 2*
Rs_Gly));
print(sprintf("r_min (flux)    = %.3f    Gly  (inner muon boundary)",
r_min_flux));
print(sprintf("r_min (cosm.)   = %.3f    Gly  (first star epoch)",
r_min_cosm));
print(sprintf("G (SI)          = %.5e  m", G_val));
print(sprintf("C               = %.5f   m^-2", C_val));
print(sprintf("R_H             = %.5f  Gly  (Hubble horizon)", RH_Gly));
print(sprintf("H0 (derived)    = %.4f   km/s/Mpc", evalf(H0_derived_kms, 5)
));
print(sprintf("Integral 0.431->16.419 = %.2f Gly  (target: 93)", evalf
(cosm_integral, 5)));
print("============================================================");

# ------------------------------------------------------------------------
---
# END OF TRANSCRIPT
# ------------------------------------------------------------------------
---
# Author:  Viorel-Mihai Popescu
# Theory:  TauTimp / Teoria Onduscularа
```

```
# Date:    May 2026
# ------------------------------------------------------------------------
---
```

"Rs (Gly) =", 8.20916

"2Rs (Gly) =", 16.4183

"Hubble horizon R_H (Gly) =", 14.5052

"Radial ODE:", $\frac{d^2}{dr^2}\, g(r) + \frac{2\left(\frac{d}{dr}\, g(r)\right)}{r} + \frac{lambda_n^2\, r\, g(r)}{Rs} = 0$

"Airy equation: d^2 phi/d xi^2 = xi * phi"

"Airy zeros a_1 = -2.3381, a_2 = -4.0879, a_3 = -5.5206"

"lambda_1 = 2.028397e-13 s^-1 m^-1"

"omega_1 = 6.080982e-05 rad/s"

"lambda_2 = 4.689308e-13 s^-1 m^-1"

"omega_2 = 1.405819e-04 rad/s"

"lambda_3 = 7.359322e-13 s^-1 m^-1"

"omega_3 = 2.206269e-04 rad/s"

"P(r,0) =", $C_norm\,(r - 0.290)\,(16.41831320 - r)$

"G (SI, m) =", 3.328537×10^{26}

"Hubble horizon R_H (SI, m) =", 1.37229×10^{26}

"C =", 5.18088, "m^-2"

"p2 =", −5.1809

"p1 =", 86.564, "Gly^-1"

"p0 =", −24.668, "Gly^-2"

"Lambda(r) =", 43.1072, "/r^2 +", −0.6533, "/r +", −0.12012

"Integral of Lambda(r) dr = -43.10796/r +", −0.6533, "*ln(r) +", −0.12012, "*r + const"

"Integral from 0.431 to 16.419 Gly =", 93.093, "Gly"

"H0 (derived) =", 67.410, "km/s/Mpc"

"P(r,t) = C * [(r - r_min)(2Rs - r) - c^2 * t^2]"

"P(r,0) check at r=8 Gly:", 336.26

"P(2Rs,0) check (BC):", 0.

Figure 1: Muon Neutrino Flux P(r,0) — Bell Profile

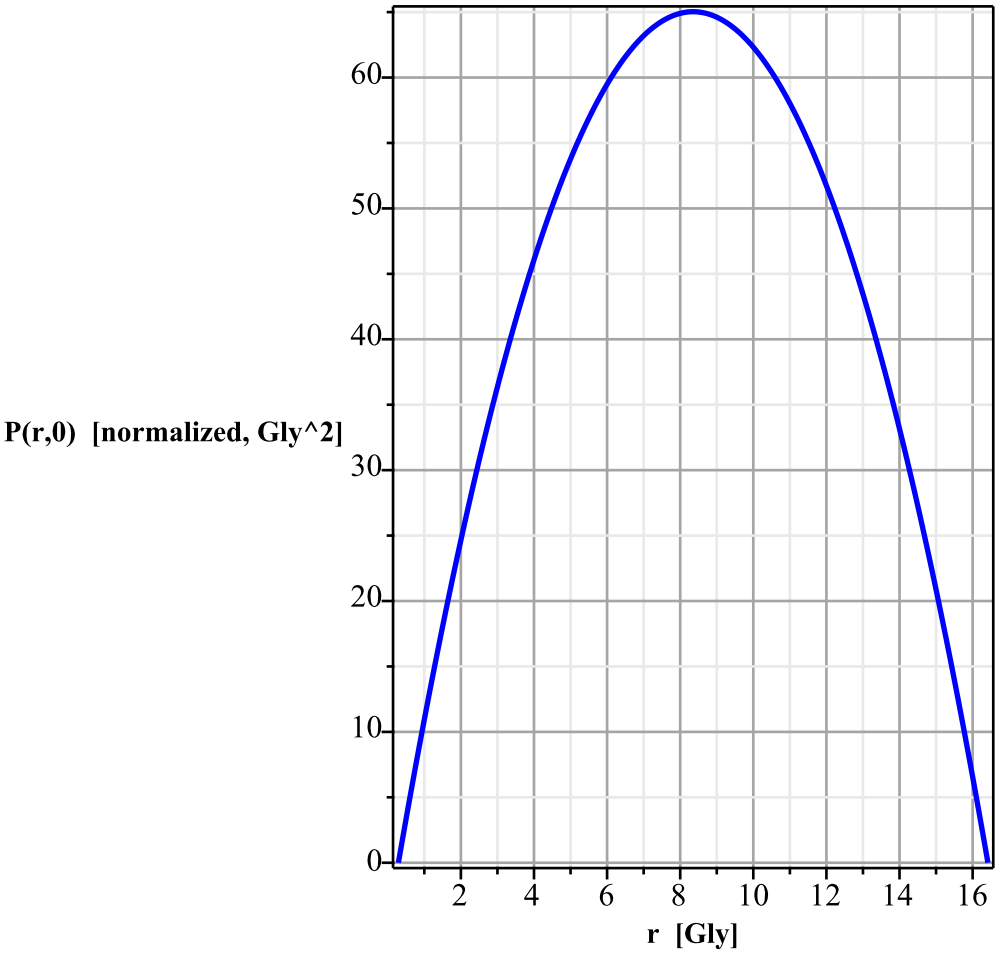


Figure 2: Spatial Flux Growth Rate Lambda(r) — TauTimp Redshift

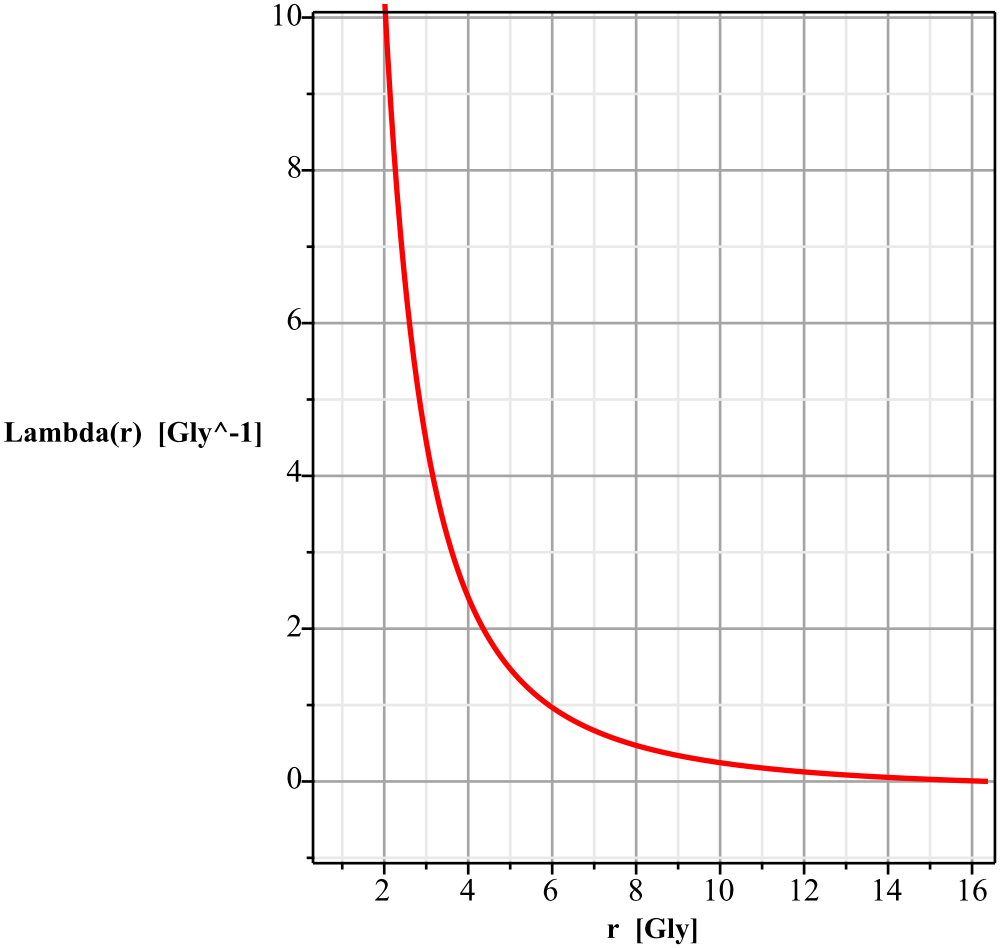


Figure 3: Cumulative Comoving Distance = Integral of Lambda(r)

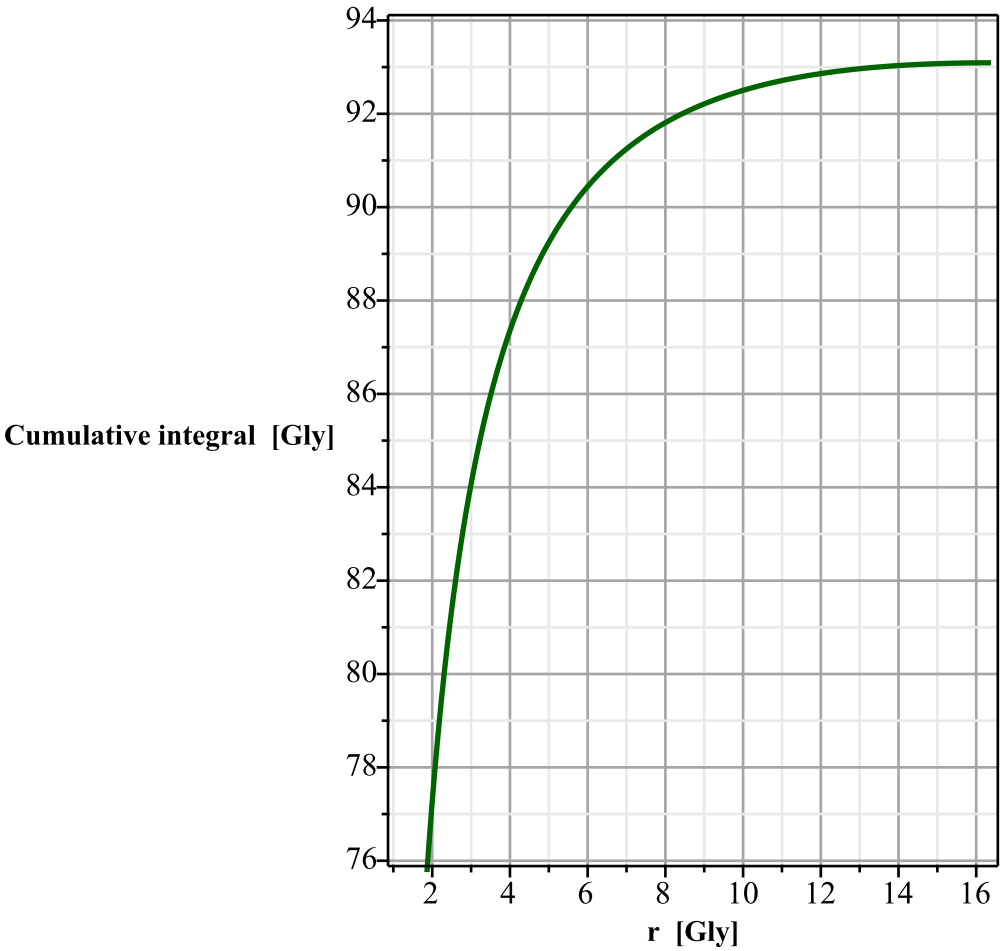
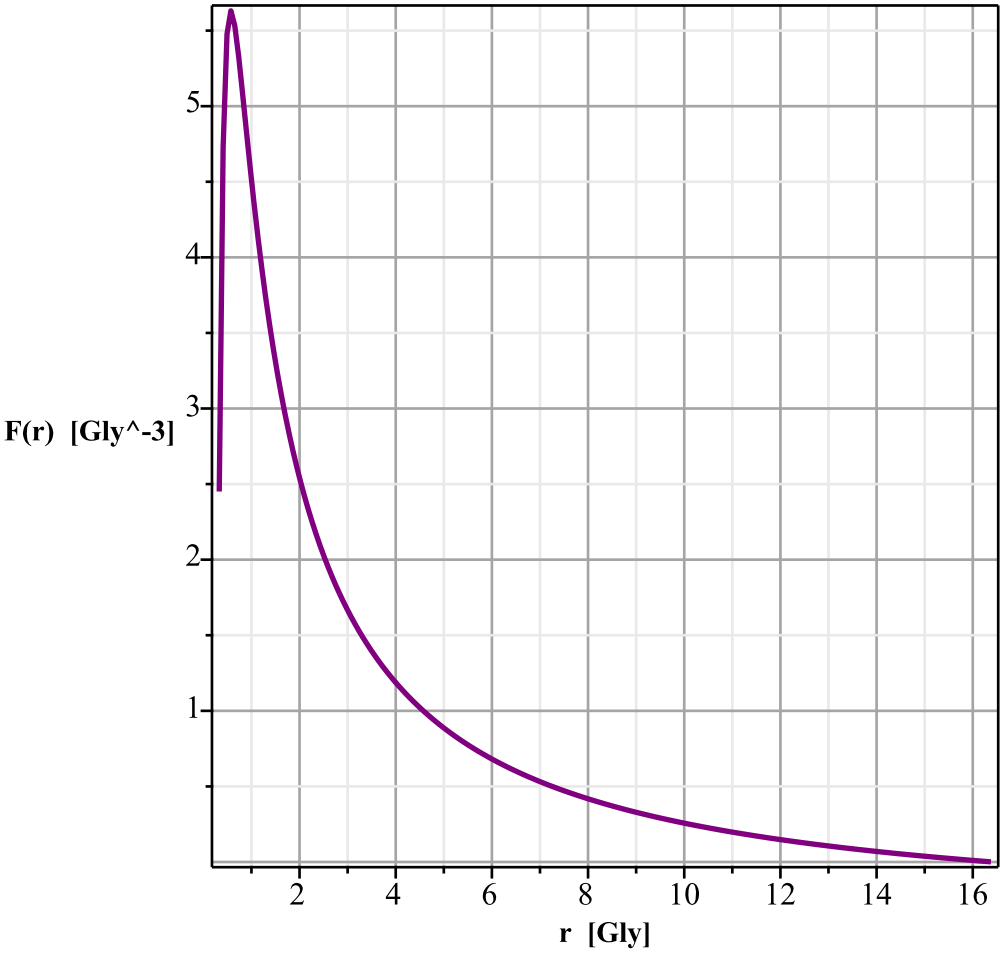


Figure 4: Onduscular Surface Flux Density F(r) = P(r,0)/(4pi r^2)



"H0 (Gyr^-1) =", 0.06894

Figure 5: Lambda(r) vs Linear Hubble Law

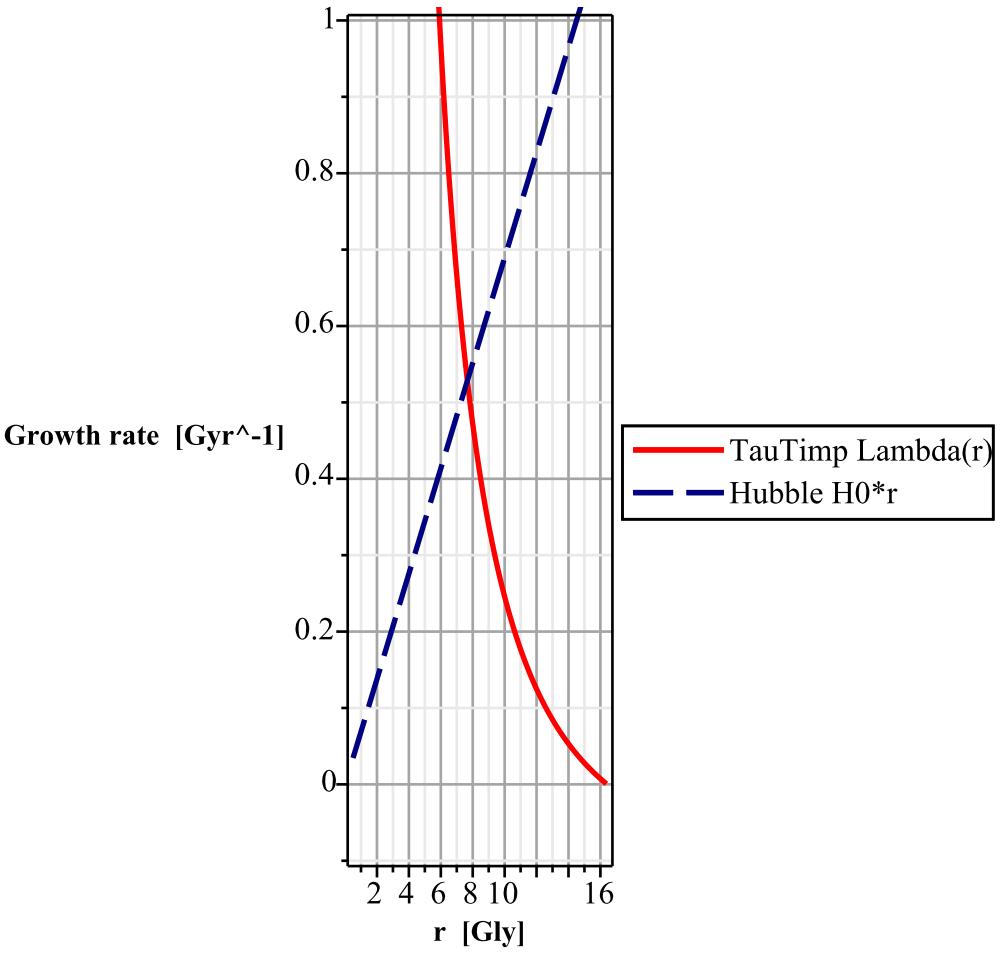
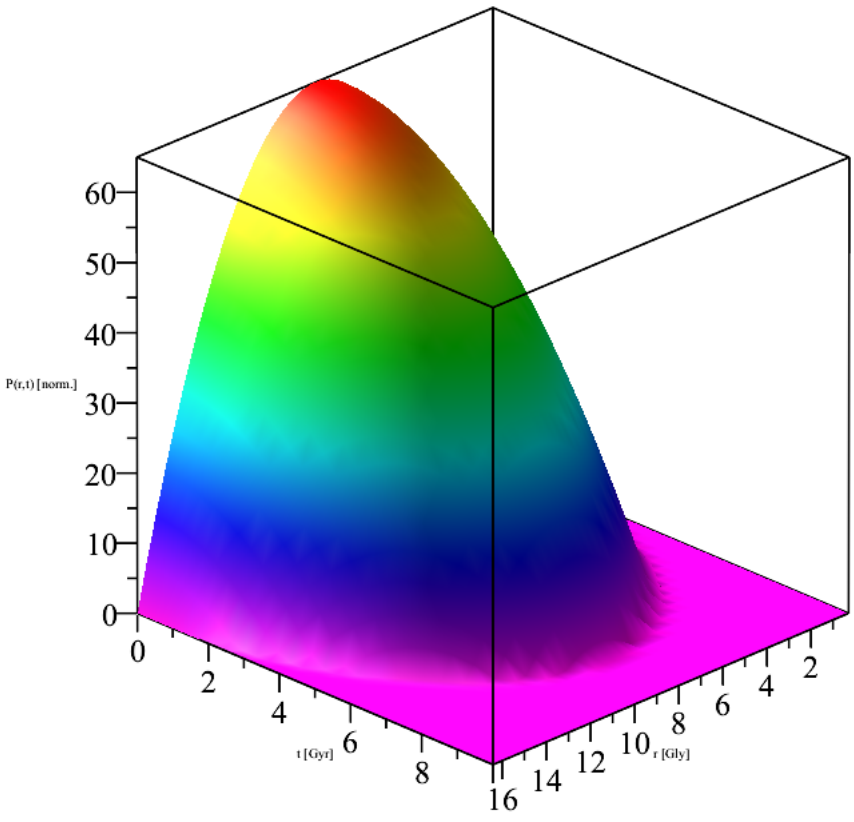
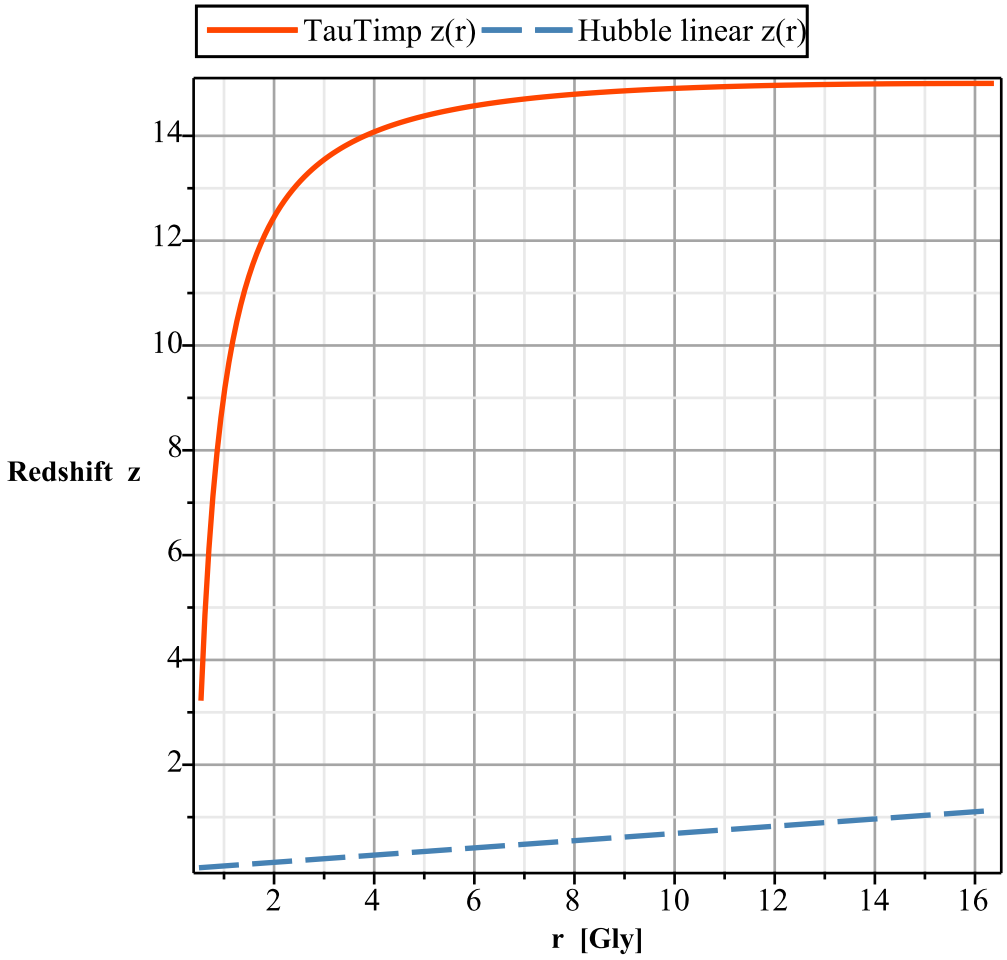


Figure 6: Time Evolution of Muon Neutrino Flux P(r,t)




```
Integral_tot (Gly)      = 93.0931
z_norm_factor (Gly^-1) = 0.161129    [= z_max / integral_tot]
```

Figure 7: Redshift z(r) — TauTimp vs Linear Hubble



"==========="

"NUMERICAL SUMMARY — TauTimp Redshift (May 2026)"

"==========="

"Rs = 8.209157 Gly"

"2Rs = 16.418313 Gly (onduscular boundary)"

"r_min (flux) = 0.290 Gly (inner muon boundary)"

"r_min (cosm.) = 0.431 Gly (first star epoch)"

"G (SI) = 3.32854e+26 m"

"C = 5.18088 m^-2"

"R_H = 14.50515 Gly (Hubble horizon)"

"H0 (derived) = 67.4100 km/s/Mpc"

"Integral 0.431->16.419 = 93.09 Gly (target: 93)"

"==========="

(1)